47th AIAA Aerospace Sciences Meeting Including The New Horizons Forum and Aerospace Exposition
5 - 8 January 2009, Orlando, Florida

AIAA 2009-1635

The Monotonic Lagrangian Particle Grid: A Fast Tracking Methodology for Air-Traffic Modeling

Carolyn Kaplan[1], Elaine Oran[2], Natalia Alexandrov[3]*, and Jay Boris[4]
Naval Research Laboratory, Washington DC 20375

**NASA Langley Research Center, Hampton VA 23681*

We are developing a dynamic global particle model to be used in a test-bed framework for evaluating approaches to global air systems control and optimization. The underlying algorithm is the Monotonic Lagrangian Grid (MLG), which is a data structure for storing and ordering particle positions and other data needed to describe *N* moving bodies. Particles that are close to each other in physical space are always near neighbors in the MLG data arrays, resulting in a fast nearest-neighbor interaction algorithm that scales as the number of particles. We present results from a series of test problems in which 4900 aircraft travel over a 2500 mi x 2500 mi area. In some scenarios, the aircraft travel a straight path to their final destination; in others, the aircraft must circumvent a restricted area. We test various empirical rules for the aircraft to avoid a no-fly zone and quantify the delay time for each case. Results indicate that delays due to collision avoidance only are relatively insignificant. When aircraft are rerouted to avoid a restricted area, the rules which govern the detour significantly affect the time in which the aircraft arrive at their final destination.

Nomenclature

N	=	the number of particles in the system
x, y, z	=	coordinates of the particles in 3D
i, j, k	=	indices of the particles in the data structure
Nx, *Ny*, *Nz*	=	the number of particles in directions *x, y, z,* respectively

I. Introduction

CONTROL and optimization of the air traffic system is vital to national economy and homeland security. In order to develop strategies for effective design and control of complex aspects of the air transportation, we must have a fast research tool for modeling various layers of the air-traffic system. We are currently developing a dynamic global particle model to be used for simulating the air traffic flow in a test-bed framework for evaluating approaches to global air systems control and optimization. The model can quickly track and sort positions of more than 10000 aircraft, both on the ground (at airports) and in the air. It will be used as an aid for testing various control strategies, such as collision avoidance, as well as evaluating the performance of the system (e.g., system's reaction to local and global perturbations) in the context of new system concepts under evaluation. For example, it could be used to determine the most efficient way to reroute air traffic after local conditions, such as thunderstorms, have greatly perturbed the entire system.

The underlying algorithm and data structure is the Monotonic Lagrangian Grid, or MLG[1-3]. The MLG has been used for two decades as the underpinning for other particle dynamics simulations, including molecular dynamics[4],

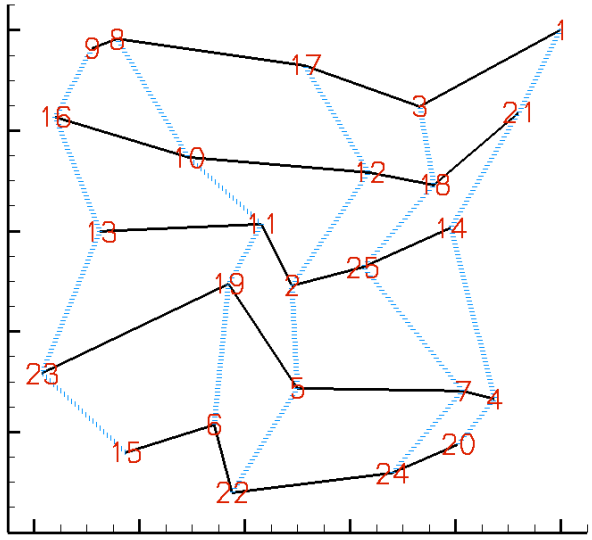
[1] Research Chemical Engineer, Laboratory for Computational Physics & Fluid Dynamics (LCP&FD), Code 6410, AIAA Associate Fellow.
[2] Senior Scientist for Reactive Flow, LCP&FD, Code 6404, AIAA Fellow.
[3] Senior Research Scientist, Aeronautics Systems Analysis Branch, Mail Stop 442, AIAA Senior Member.
[4] Chief Scientist and Director, LCP&FD, Code 6400, AIAA Fellow.

direct simulation Monte Carlo[5] and missile defense[6-9] applications. A three-dimensional MLG data structure is defined by the constraints:

$$\begin{array}{lllll} x(i,j,k) \le x(i+1,j,k) & i=1,....,Nx-1; & j=1,.....,Ny; & k=1,.....,Nz \\ y(i,j,k) \le y(i,j+1,k) & i=1,....,Nx; & j=1,...,Ny-1; & k=1,....,Nz \\ z(i,j,k) \le z(i,j,k+1) & i=1,....,Nx; & j=1,.....,Ny; & k=1,.....,Nz-1, \end{array} \quad (1)$$

where *Nx*, *Ny,* and *Nz* are the number of particles in each direction. The constraints mean that each grid line in each spatial direction is forced to be a monotone index mapping.

Figure 1 depicts an example of a small two-dimensional MLG, showing the locations of 25 aircraft at one point in time. The particle model developed here is three-dimensional in space; however we are showing a two-dimensional subset to explain the principles of the MLG. The nodes are shown at their irregular spatial locations, but they are indexed regularly in the MLG by a monotonic mapping between the grid indices and the locations. The table shows the grid indices (in *i-j* space) of each aircraft.



5	9	8	17	3	1
4	16	10	12	18	21
3	13	11	2	25	14
2	23	19	5	7	4
1	15	6	22	24	20
J/I	1	2	3	4	5

Figure 1. An example of a two-dimensional MLG. *The figure shows a 2-D MLG (5x5) containing the x- and y-locations of 25 labeled aircraft. The solid black (horizontal) lines show the x-links and the dotted blue (vertical) lines show the y-links. The table shows the regular grid indices of the nodes shown in the figure. That is, aircraft-15 is indexed at i = 1, j =1; aircraft-6 is indexed at i =2, j = 1, etc.*

The MLG is a free-Lagrangian data structure for storing the positions and other data needed to describe *N* moving bodies. A node (point in space) with three spatial coordinates has three indices in the MLG data arrays. Data relating to each node are stored in computer memory locations defined by these indices. Thus nodes which are close to each other in physical space are always near neighbors in the MLG data arrays. A computer program based on the MLG data structure does not need to check *N-1* possible distances to find which nodes are close to a particular node. The indices of the neighboring nodes are automatically known because the MLG node indices vary monotonically in all directions with the Lagrangian node coordinates. For example, as shown in Figure 1, we automatically know that near neighbors of aircraft-11 are aircrafts-16, 10, 12, 13, 2, 23, 19, 5, without having to check the distances of all 24 remaining aircraft. The cost of the tracking algorithm is dominated by the calculation of the interactions of nodes with their near neighbors, and the timing thus scales as *N*.

When putting nodes in MLG order, the MLG algorithm sorts each axis individually. That is, for a 5x5 MLG, it sorts the first five points in the x-direction, then the next five points in the x-direction, etc. After all 25 points have been sorted in the x-direction, it then sorts all 25 points in the y-direction. As each axis becomes sorted, the sorting process may destroy monotonicity in the other axes. So, the sorting process is repeated until all axes are monotonic. The MLG uses two sorting algorithms to put nodes in order. One algorithm is a bubble-sort, in which each node is compared with the one following it, and if they are not in monotonic order, they are switched. This bubble-sorting

algorithm is best used when the nodes are already partially pre-sorted. The other algorithm is a shell-sort, in which each node is compared with one that is a half-axis length away, and if the two nodes are not in monotonic order, they are switched. This shell-sorting algorithm is best for completely random data.

When using the MLG algorithm, the user can specify the sort direction (sort from left to right, or from right to left), the axis order (sort x-axis first, then y-axis, or the reverse), and the sorting algorithm (bubble or shell sort). All of these factors affect the MLG that is obtained. That is, the MLG is not unique[10, 11]. Table 1 shows the number of possible MLGs that can be obtained for a given number of nodes. As shown in the table, even for a small number of nodes, such as 16, there can be up to 405 possible MLGs.

No. of Nodes	MLG Shape	No. of possible MLGs
4	2x2	1 - 2
9	3x3	3 - 12
16	4x4	91 - 405
25	5x5	10130 - 97799

Table 1. The number of possible MLGs as a function of the number of nodes. The MLG obtained for a given number of nodes is not unique, and some MLGs are of higher quality than others.[10,11]

Figure 2 shows three different MLGs based on the same set of node locations. Image (a) shows an MLG obtained when sorting nodes from a random order. This is a valid MLG, as it satisfies the constraints in Eq 1. However, this is a poor quality MLG, because it is tangled, and therefore nodes which are nearest-neighbors in *i-j* space may actually be far apart in physical space. In order to gauge the quality of an MLG, we have developed a set of diagnostics that is used during a simulation. The diagnostics are based on a dot product which indicates how close an x-link is to the horizontal, and how close a y-link is to the vertical. A perfectly orthogonal MLG would have a dot product equal to unity for all x-links and y-links. Obviously, an MLG based on real data (such as positions of aircraft in the air) would never form a perfectly orthogonal MLG. However, the MLG algorithm uses a grid restructuring technique, called Stochastic Grid Regularization (SGR) to choose an MLG with the highest quality[10, 11]. The SGR technique is a simulated annealing process, in which the nodes are randomly perturbed, and an MLG of the perturbed nodes is used as a starting point for creating an MLG of the original (unperturbed) nodes. This technique can be applied locally to the parts of the MLG which are tangled, and it does not significantly increase the cost of the overall calculation. Images (b) and (c) in Fig 2 show how the SGR technique restructures the tangled grid after one and two iterations of SGR, respectively.

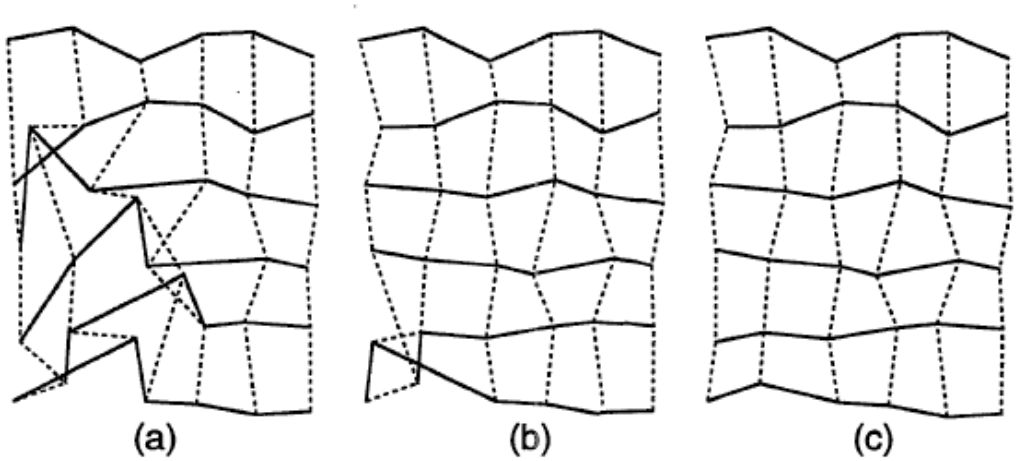


Figure 2. Three different MLGs based on the same set of node locations. *Image (a) is an MLG obtained when sorting nodes from a random order. Image (b) is the same MLG after one iteration of SGR, while image (c) is the same MLG after two iterations of SGR.*

II. Methodology

We have used the MLG in our global particle model as a method for ordering and tracking locations of many aircraft for various test case scenarios. Although the model is 3-D, we present results here for 2-D cases, to facilitate visualization. The overall methodology of the global particle model tracking consists of the following steps:

1. Read the flight plan and assign each aircraft: an ID, an initial *x-, y-, z*-location, a final *x-, y-, z*-location, and average speed;
2. Calculate an initial velocity vector for each aircraft, assuming a straight path between its initial and final destinations;
3. Put data in MLG data structure (ID and current x-, y-, z-location);
4. Sort aircraft into MLG order, based on x-, y- and z-location;
5. Do diagnostics to check the quality of the MLG;
6. Avoid restricted area (e.g., bad weather in a no-fly zone) as follows:
   Project forward in time for each aircraft. If the projected flight path is expected to enter the restricted zone, calculate a new velocity vector. The amount of time projected forward and the direction of the new velocity vector depends on the particular test scenario.
7. Avoid collisions as follows:
   Check nearest neighbors for potential collisions. If aircraft are within five miles of each other (approximately 30 seconds apart), then project forward in time for 10 seconds. Then, if aircraft are still within five miles of each other, modify the velocity vector to avoid collision. The amount that the velocity is modified depends on the relative velocities and positions of the aircraft.
8. Move aircraft, based on velocity vector computed above.
9. At one minute intervals, recalculate the velocity vector for each aircraft, so that there is a straight path to final *x-, y-, z*-locations. This step is periodically necessary because the aircraft have been detoured in steps #6 and #7, to avoid the restricted area and to avoid collisions.
10. Increment time, and go back to step #4.

This is the present tracking strategy. Different strategy components (e.g., a variety of collision avoidance techniques) can be incorporated in the methodology.

III. Results

We have conducted a series of tests in which 4900 aircraft travel over a 2500 mile x 2500 mile area, a scenario analogous to having 4900 aircraft over an area equivalent to the United States. Initially, half of the aircraft are randomly placed on the east side of the domain and are randomly assigned a final destination on the west side; the other half are randomly placed on the west side of the domain and are randomly assigned a final destination on the east side. A schematic of this test problem is shown in Figure 3. An average speed for each aircraft is assigned randomly, ranging from 400 to 500 miles per hour.

A. Scenario 1 – straight path to final destination

In the first test scenario, the aircraft travel along a straight path to the other side, and the only detours allowed are to avoid collisions between aircraft. Figure 4 shows a time sequence from zero to five hours of physical time. At time zero, half of the aircrafts are on the east side, and half are on the west side. After one hour of physical time (image (b)), the two groups are moving towards each other, and the spread within each group is due to the difference in average speed and velocity vector among the aircraft. After two hours, the two groups have merged at the center of the domain, and the number of collision-avoidance moves is at its peak. By three hours, the groups have begun to separate, as the aircraft move to opposite sides of the domain, but there is still a cluster of aircraft from both groups at the center of the domain. After four hours (image (e)), the groups have completely separated, and by six hours (not shown), most aircraft have reached their final destination.

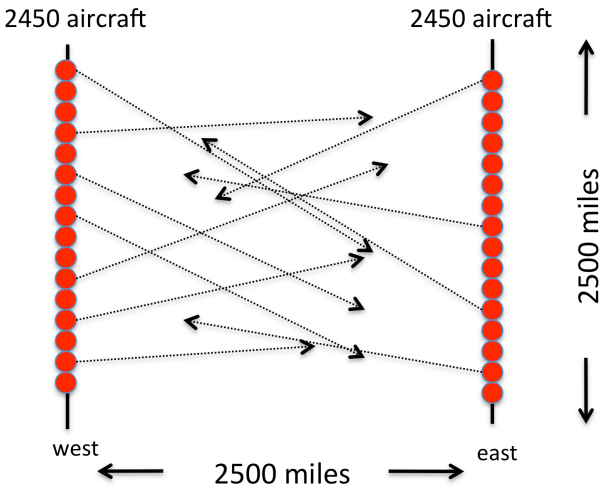


Figure 3. Simplified schematic diagram of the test problem. *The full scenario includes 4900 aircraft, of which half are randomly placed along the east border, and the other half are randomly placed along the west border. Each aircraft is randomly assigned a final destination on the other side of the domain, and is also randomly assigned an average speed ranging from 400 to 500 miles/hr.*

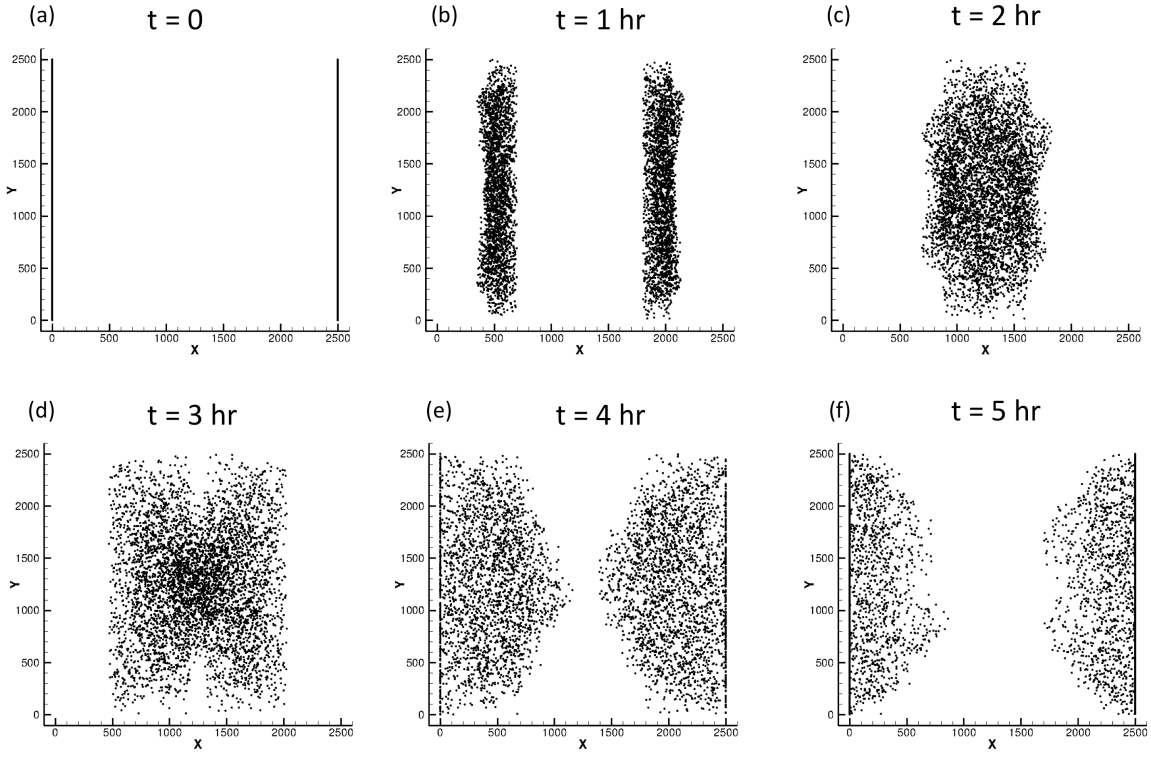


Figure 4. Time sequence of the first scenario, in which aircrafts travel in a straight path from one side to the other. The only detours allowed are for collision avoidance between aircraft. *Image (a) corresponds to time zero (half of the aircraft on the east side, and half on the west side). The other images (b through f) are shown at intervals of one hour of physical time. By six hours (not shown), practically all aircraft have reached their destination on the other side of the domain.*

During the simulation, we track the number of the collision-avoidance moves for each aircraft, and each aircraft's arrival time at its final destination. We also calculate an ideal arrival time, i.e., the time the aircraft takes to travel in a straight path to final destination, without making detours to avoid collisions. Figure 5 shows the delay time (actual minus ideal arrival time) in minutes for each aircraft, as a function of the number of collision avoidance moves. As expected, the delay time for each aircraft increases with the number of collision-avoidance maneuvers, with a significant amount of statistical scatter among the 4900 aircraft. In practically all cases, the delay time is less than 10 minutes, which is insignificant compared to flight times of five to six hours.

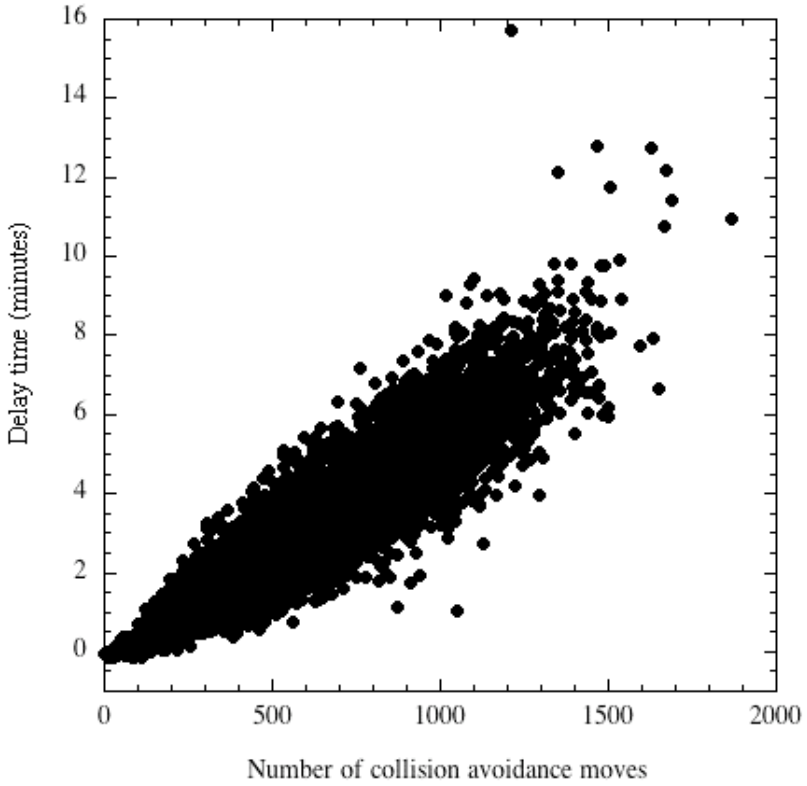


Figure 5. For Scenario 1, the delay times, defined as the actual arrival time minus the ideal arrival time, as a function of the number of collision-avoidance moves for each aircraft.

B. Scenario 2 – Avoiding a restricted area in flight path

In the second test case, there is a restricted area through which aircraft are not permitted to fly. This could, for example, represent an area of bad weather, or an imposed no-fly zone. As shown in Figure 6, the restricted area measures 100 mi x 100 mi and is located in the center of the computational domain. In these cases, only those aircrafts whose paths are projected to go through the restricted area are re-outed around it. All other aircraft, whose trajectories do not pass through this area, follow their intended paths. Figure 6 also shows an instantaneous image from one of the Scenario 2 test cases. These simulations are are performed with the same rules and initial conditions as used in Scenario 1, except for the presence of the restricted area, which must be circumvented by all aircraft projected to pass through it.

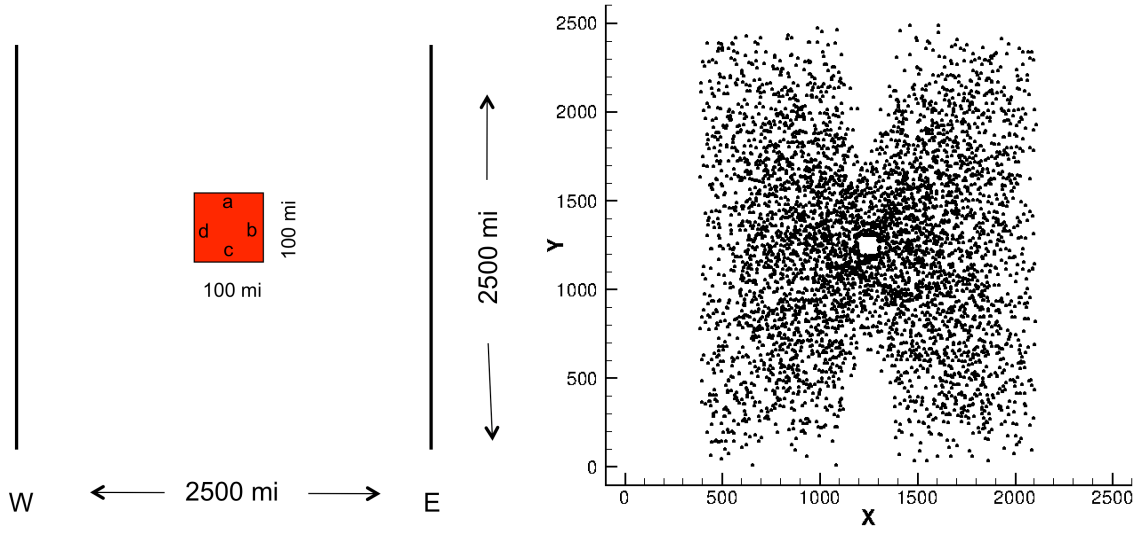


Figure 6. Scenario 2 conditions. *The left side shows a schematic for Scenario 2 test cases in which there is a restricted zone in the center of the computational domain. The right side shows an instantaneous image from one of the Scenario 2 test cases.*

Figure 7 shows three sets of "rules" to govern how the aircraft circumvent the restricted zone. In cases (a) and (b), the existence of the restricted zone occurs suddenly (e.g., an unexpected thunderstorm), and aircraft must abruptly circumvent the region when they are within two minutes of the area. In case (c), the pilot has some advance warning to plan a more direct route around the restricted area.

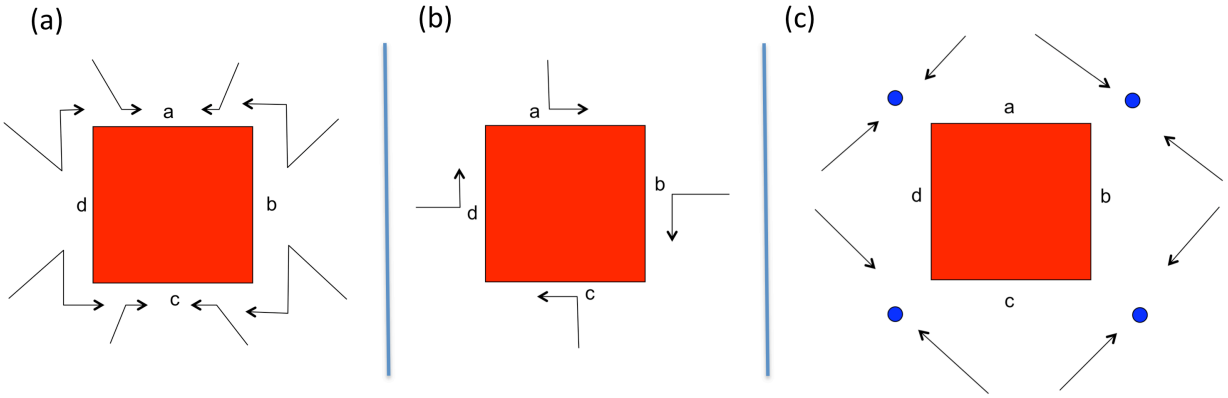


Figure 7. Three sets of rules to govern how the aircraft circumvent the restricted zone.

In case (a), the aircraft that are projected to enter through *face a* are routed as shown by the two arrows at the top; that is, aircraft that enter from the northeast (and are headed west) are routed to the left, while aircraft that enter from the northwest (and are headed east) are routed to the right. Likewise, aircraft that would enter through *face b* are routed as shown by the two arrows at the right; that is, aircraft that enter from the northeast (and are headed west) are routed up and to the left, while aircraft that enter from the southeast (and are headed west) are routed down and to the left. As shown by all of the arrows in Fig 7a, aircraft that are headed west travel to the left and aircraft that are headed east travel to the right. If the aircraft is coming from the north, it travels over the top, and if it is coming from the south, it travels around the bottom.

In case (b), the aircraft circumvent the restricted area by traveling in a clockwise direction. For example, aircraft which are projected to intersect *face a* are routed to the right and continue in a clockwise direction around the restricted zone until it can proceed in a straight line path to its final destination.

Lastly, in case (c), the aircraft have advanced warning about the presence of the restricted area. The aircraft are rerouted to specific way-points (blue dots) 15 minutes before they would have arrived in the restricted zone. As shown in Fig 7c, aircraft coming from the southeast are directed to the blue dot in the upper right-hand-corner, while those coming from the northeast are directed to the blue dot in the lower right-hand-corner.

During these simulations, we track the arrival time of each aircraft at its final destination. The delay time is then calculated to be the arrival time minus the arrival time if the aircraft had traveled a straight path (if it had not avoided the restricted area). Figure 8 shows the delay time versus the number of flights for the three cases. Image (a) shows results out to 10 minutes of delay time, while image (b) shows results from 10 to 30 minutes of delay time. As shown, case (c) results in the most aircraft having short delay times. This is an expected result, as the aircraft in this case began their detours 15 minutes in advance. There was not much difference between cases (a) and (b). This was somewhat surprising, as we would expect that having all aircraft traveling in the same direction (as in case (b)) would have resulted in shorter delay times. As shown image (b), case (c) resulted in significantly fewer aircraft having longer delay times, as expected.

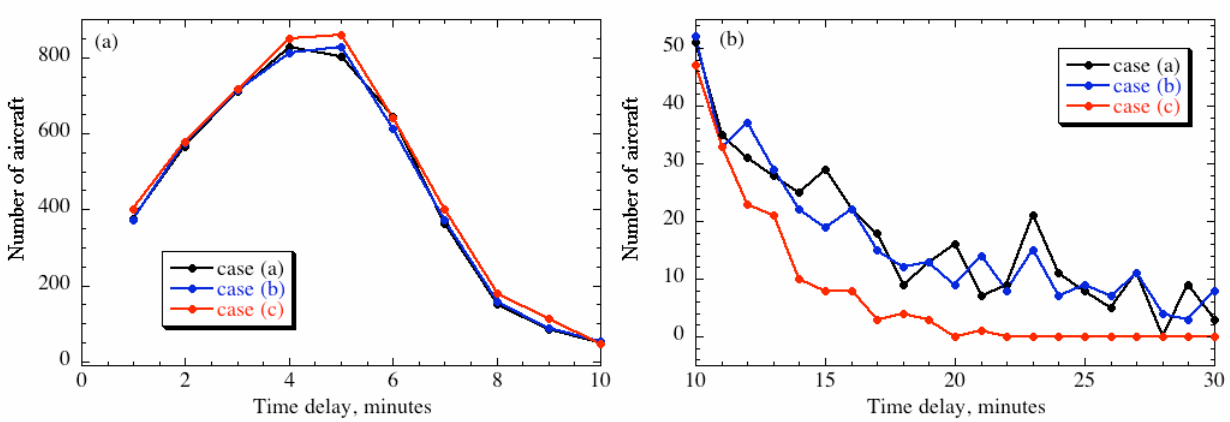


Figure 8. Delay time versus number of aircraft for (a) short delay times (less than 10 minutes), and for (b) longer delay times (10-30 minutes).

Lastly, we consider the effect of enlarging the size of the restricted area to 500 mi x 500 mi. This last scenario used the clockwise flow rule (case (b)) to avoid the restricted area. As expected, the larger restricted area results in more aircraft delayed for 15 or more minutes, as shown in Fig 9.

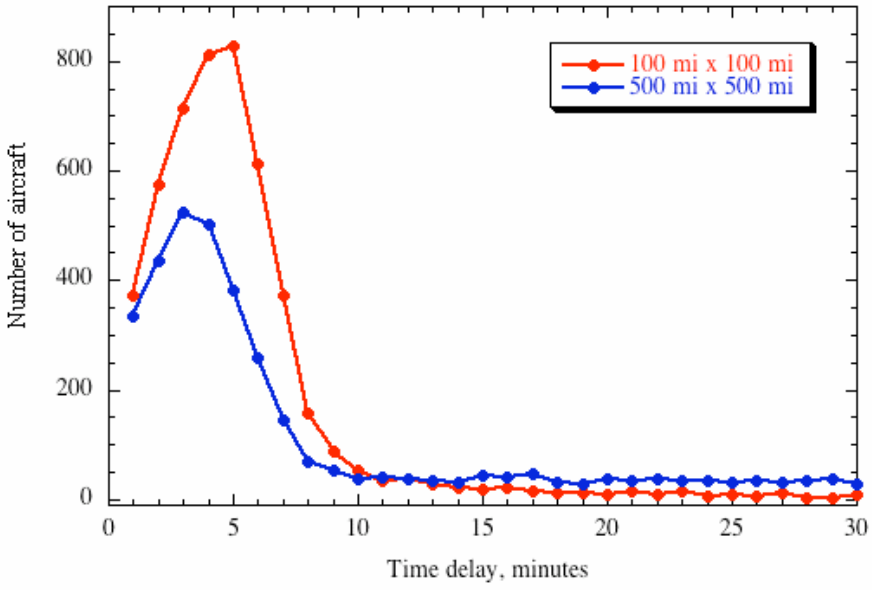


Figure 9. Effect of size of the restricted area size on delay time.

IV. Discussion and Concluding Remarks

In summary, we have developed a dynamic global particle model for air traffic, to be used in a computational test-bed framework for evaluating approaches to global air systems control and optimization. The underlying algorithm used in the model is the MLG, which can quickly track and sort positions of many aircraft, both on the ground (at airports) and in the air. By using the fast sorting algorithm in the MLG, the simulations presented here (with 4900 aircraft) have used approximately 10 minutes of CPU time, on a single processor machine, to simulate 10 hours of physical time.

The model is currently being used as a test-bed for evaluating the system's reaction to local and global perturbations. We have shown a series of test problems based on 4900 aircraft traveling over a 2500 mi x 2500 mi area, and have simulated two scenarios: one in which aircraft travel a straight path from initial to final destination (detours occur for collision avoidance only), and the other in which aircraft are required to avoid a restricted area in the center of the computational domain. Results indicate that delays due to collision avoidance only are relatively insignificant. However, when aircraft are rerouted to avoid a restricted area, the rules which govern the detour significantly affect the time at which the aircraft arrive at their final destination.

A few words are in order about the motivation and context of using the MLG model. Our major target is the investigation of systemic phenomena in complex air traffic systems, including the so-called emergent (unpredicted) effects. Both the study of emergent effects and the active control and optimization of a complex system require the derivation of the functional dependences of the global characteristics of interest (e.g., throughput and capacity) on the local parameters of a complex transport network. Model derivation requires massive computational experiments. Thus, flexible and *fast* simulation, assisted by parallel and distributed computations, facilitates model building to a great extent. This is why, despite the availability of sophisticated air traffic simulators (e.g., FACET)[12], we are interested in developing a fast simulator that can be easily incorporated into the overall computational framework for studying complex transport systems[13]. We plan to use the existing simulators, in addition to the MLG tool, for model validation.

Future plans include incorporating the particle model into the framework, along with a variety of network topology scenarios and optimization strategies, to derive functional dependences of global characteristics (throughput or capacity) on the local parameters of the network for system design and optimization studies.

Acknowledgments

The authors gratefully acknowledge support from NASA Langley Research Center (LaRC) under Interagency Agreement IA1-910 between LaRC and LCP&FD.

References

[1]Boris, J., "A Vectorized "Near Neighbors" Algorithm of Order N Using a Monotonic Logical Grid," *J. of Computational Physics*, **66**, 1 (1986).

[2]Picone, J.M., Lambrakos, S.G., Boris, J.P. and Jajodia, S., "Initial Comparison of Monotonic Logical Grid and Alternative Data Base Structures," NRL Memorandum Report 5860, Sept 30, 1986.

[3]Lambrakos, S.G. and Boris, J.P., "Geometric Properties of the Monotonic Logical Grid Algorithm for Near Neighbor Calculations," J. of Computational Physics, **73**, 183 (1987).

[4]Lambrakos, S.G., Nagumo, M., Boris, J.P., Oran, E.S., and Gaber, B., "Molecular Dynamics Simulation of a Lipid Bilayer Using Novel Monotonic Logical Grid (MLG) and Multiple Constraint Relaxation (MCR) Algorithms," Biophysical Journal, **51**, A440 (1987).

[5]Cybyk, B., Oran, E.S., Boris, J.P., and Anderson, J.D., "Combining the Monotonic Lagrangian Grid with a direct simulation Monte Carlo Model," J. of Computational Physics, **122**, 323 (1995).

[6]Boris, J.P., Picone, J.M., and Lambrakos, S.G., "BEAST: A High-Performance Battle Engagement Area Simulator/Tracker," NRL Memorandum Report 5908, December 31, 1986.

[7]Jajodia, S., Meadows, C.A., Boris, J.P., Picone, J.M., and Lambrakos, S.G., "NRL Research in Database Management for the SDI Battle Management System," NRL Memorandum Report 5921, January 30, 1987. Distribution limited.

[8]Picone, J.M., Boris, J.P., Lambrakos, S.G., Uhlmann, J. and Zuniga, M., "Near-Neighbor Algorithms for Processing Bearing Data," NRL Memorandum Report 6456, May 10, 1989.

[9]Kolbe, R.L., Boris, J.P. and Picone, J.M., "Battle Engagement Area Simulator/Tracker," NRL Memorandum Report 6705, October 8, 1990.

[10]Sinkovits, R.S., Oran, E.S. and Boris, J.P., "A Technique for Regularizing the Structure of a Monotonic Lagrangian Grid," J. of Computational Physics, 108, 368 (1993).

[11]Sinkovits, R.S., Boris, J.P. and Oran, E.S., "The Stability and Multiplicity of the Monotonic Lagrangian Grid," NRL Memorandum Report 6410-97-7937, 1997.
[12]Bilimoria, K., Sridhar, B., Chatterji, G., Sheth, K., Grabbe, S.: FACET: Future ATM Concepts Evaluation Tool, 3rd USA/Europe Air Traffic Management R&D Seminar, Napoli, Italy, June 2000.
[13]Alexandrov, N., Kincaid, R.K., Vargo, E.P., "Toward Optimal Transport Networks", AIAA Paper 2008-5814, American Institute of Aeronautics and Astronautics, Reston, VA, 2008.